



Celeris

SysMLv2 Mermaid Viewer

Version 0.5.0 · User Manual

Features

- Render the contents of a SysML v2 package as a Mermaid diagram from the editor (right-click → Celeris).
- Seven diagram types: flowchart, class, state, sequence, requirement, interconnection (IBD) and realization.
- Realization view: cross-level traceability — elements placed in engineering-level lanes (Arcadia/NAF) with realization dependencies drawn between levels.
- Tabular views: requirements, interfaces (ports / parts / flows) and any element type as a table, configurable via a .table.json (select + columns) plus built-in presets, with CSV export.
- Gantt (schedule) view: an activity sequence (actions + successions) as a Gantt — duration and resource attributes set each bar's length and colour, and the critical path is outlined in red.
- The class diagram doubles as a Block Definition Diagram (BDD): function breakdowns (action def) and component breakdowns (part def) render as composition trees.
- Interconnection (Internal Block Diagram): parts with their ports wired by connect/interface, and actions with in/out ref parameters wired by item flows.
- In-process parsing with the open-source SysML v2 parser (syside-languageserver); reusable profile libraries in sibling files are resolved automatically.
- Per-panel toolbar: ↻ Refresh, ↓ PNG export, zoom/pan controls (–, 1:1, +, Fit, mouse-wheel zoom, drag-to-pan), and a syntax-error indicator.
- Auto-refresh open diagrams when the .sysml file is saved (configurable).
- Type-based styling from a JSON style file: colour elements by native SysML type and/or by the SysML type they conform to (hierarchy-followed).
- Flowchart colour code by node kind by default (actions blue, control nodes amber, parts green), overridable by a style file.
- Performers in flowcharts: a part that performs an action is shown as a component linked to it by a dashed allocation edge (which component realises each function).
- Capella-style functional chains: model each chain as a use case that performs its functions to highlight the chain (and its flows) with bold coloured borders, with shared functions marked.
- Ready-made Arcadia and NAF v4 styling profiles.
- View toggles (Defs, Inherited) to declutter the class and requirement diagrams: hide definition boxes and/or inherited types (inheritance edges and imported supertypes), per panel.
- Open several diagrams at once, one per panel.

Highlights

- **Flowchart.** Action flows with decisions, fork/join/merge and guarded transitions.
- **Class / State / Sequence.** Three further diagram types sharing the parser and rendering infrastructure.
- **Auto-refresh & PNG export.** Live refresh on save and one-click PNG export of any diagram.
- **Type-based styling.** JSON style files, with reusable Arcadia and NAF v4 profiles.
- **Functional chains & performers.** Capella-style chains (use case + perform) and the parts that perform each function, in flowcharts; built-in colour code by node kind.
- **View toggles.** Hide definitions / inherited types in the class and requirement diagrams (per-panel toolbar).
- **Breakdown (BDD).** The class diagram doubles as a Block Definition Diagram: function and component breakdown trees via composition.
- **Interconnection (IBD).** A sixth view: parts wired by ports/connect/interface, and functions wired by in/out parameters and item flows.
- **Realization.** A seventh view: cross-level traceability — engineering-level lanes (Arcadia/NAF) with realization dependencies between them.
- **Tabular views.** Requirements / interfaces / any-element tables (HTML) with CSV export, configurable via a .table.json plus built-in presets.
- **Gantt (schedule).** Activity sequences as a Gantt: duration/resource attributes drive bar length and colour; the critical path is outlined in red.

Examples

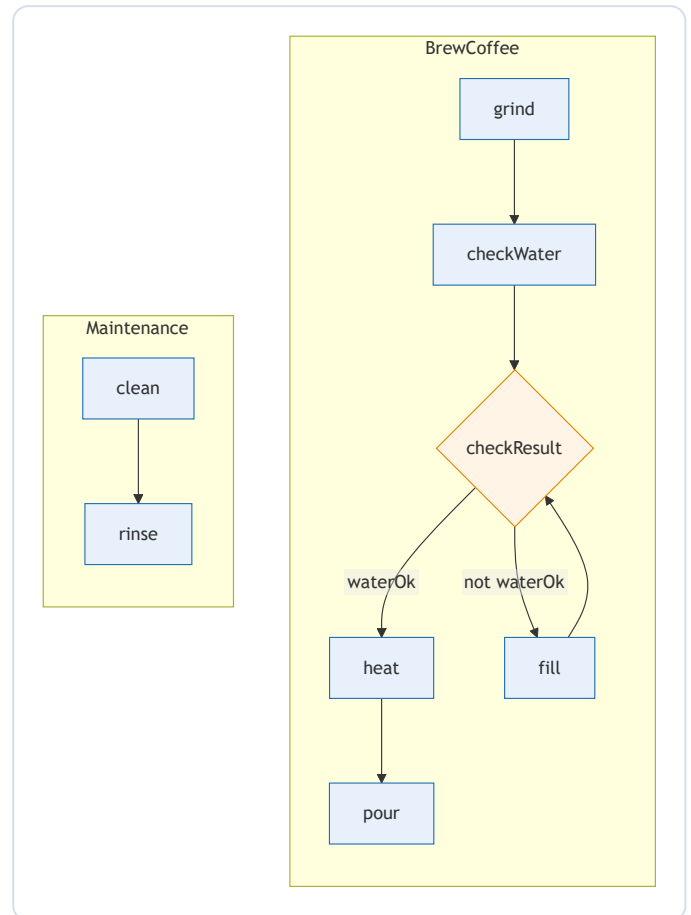
Flowchart — action flow with a decision

An action def with a decision node, guarded branches and a loop. Action usages become nodes, successions become edges, and transition guards label the edges. Non-flow elements (attributes, parts) are filtered out.

EXAMPLES/COFFEE.SYSML

```
package CoffeeMachine {  
    private import ScalarValues::*;  
    action def BrewCoffee {  
        attribute waterOk : Boolean;  
  
        action grind;  
        action checkWater;  
        decide checkResult;  
        action heat;  
        action fill;  
        action pour;  
  
        succession first grind then checkWater;  
        succession first checkWater then  
checkResult;  
        first checkResult if waterOk then heat;  
        first checkResult if not waterOk then fill;  
        succession first fill then checkResult;  
        succession first heat then pour;  
    }  
  
    action def Maintenance {  
        action clean;  
        action rinse;  
        succession first clean then rinse;  
    }  
  
    part def Tank;  
    attribute level : ScalarValues::Integer;  
}
```

FLOWCHART DIAGRAM



Flowchart — parallel actions (fork / join)

`fork` and `join` control nodes split work into parallel branches and re-synchronize them.

EXAMPLES/ORDER_FULFILLMENT.SYSML

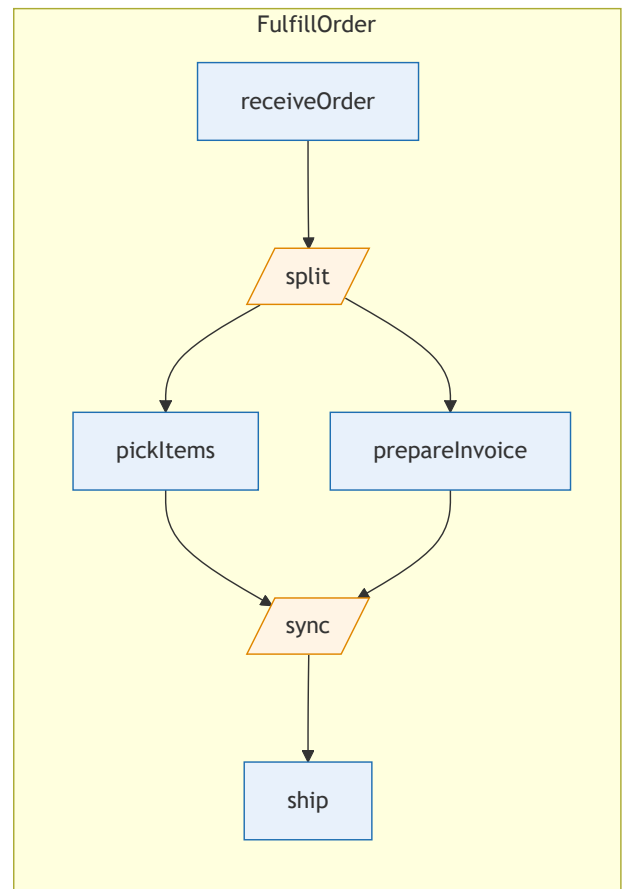
```
// Flowchart example: parallel actions with fork / join.
package OrderFulfillment {

    action def FulfillOrder {
        action receiveOrder;
        fork split;
        action pickItems;
        action prepareInvoice;
        join sync;
        action ship;

        succession first receiveOrder then split;
        succession first split then pickItems;
        succession first split then prepareInvoice;
        succession first pickItems then sync;
        succession first prepareInvoice then sync;
        succession first sync then ship;

    }
}
```

FLOWCHART DIAGRAM



Flowchart — functional chains & performers (Capella-style)

Each chain is a **use case** that performs its functions; the boxes and flows of a chain get a bold coloured border, and a function shared by several chains (here **identify**) gets a dark border. The **part** performers (green) are linked to the actions they realise by dashed *performed by* edges. Actions (blue) and parts (green) are colour-coded by default. Legends recall both codes.

EXAMPLES/CHAINS/SURVEILLANCE.SYSML

```
// Functional architecture with 3 functional chains  
(Capella-style).
```

```
// View as a Flowchart (right-click → Celeris →  
Show Flowchart Diagram):
```

```
// Engagement, Reporting and Archival each run  
through several functions; the
```

```
// 'identify' function is shared by all three  
(dark border).
```

```
//
```

```
// Chains are modelled the SysML v2 way: each chain  
is a 'use case' that
```

```
// 'perform's the functions it traverses. A  
function performed by several use
```

```
// cases is the one shared by several chains. The  
'part's are the performers:
```

```
// the components that perform (realise) each  
function.
```

```
package SurveillanceArchitecture {
```

```
    // — Functional architecture: the functions  
    and the flows between them —
```

```
    action MissionThread {  
        action sense;  
        action detect;  
        action identify;  
        action engage;  
        action report;  
        action archive;  
  
        // Functional exchanges  
  
        succession first sense then identify;  
        succession first detect then identify;  
        succession first identify then engage;  
        succession first identify then report;  
        succession first identify then archive;  
    }
```

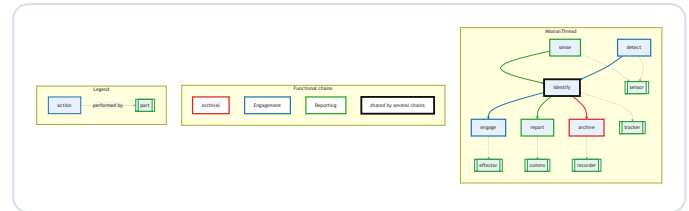
```
    // — Functional chains: each chain is a use  
    case performing its functions —
```

```
    use case def Engagement {  
        perform MissionThread.detect;  
        perform MissionThread.identify;  
        perform MissionThread.engage;  
    }  
    use case def Reporting {  
        perform MissionThread.sense;  
        perform MissionThread.identify;  
        perform MissionThread.report;  
    }  
    use case def Archival {  
        perform MissionThread.identify;  
        perform MissionThread.archive;  
    }
```

```
    // — Performers: the components that perform  
(realise) each function —
```

```
    part def Sensor;  
    part def Tracker;  
    part def Effector;
```

FLOWCHART DIAGRAM



```

part def Comms;
part def Recorder;

part sensor : Sensor {
  perform MissionThread.sense;
  perform MissionThread.detect;
}
part tracker : Tracker {
  perform MissionThread.identify;
}
part effector : Effector {
  perform MissionThread.engage;
}
part comms : Comms {
  perform MissionThread.report;
}
part recorder : Recorder {
  perform MissionThread.archive;
}
}

```

Class — structure, inheritance & composition

Definitions become classes; specializations (:>) are inheritance, typed part usages are compositions with multiplicities, and enumerations list their literals.

EXAMPLES/VEHICLE.SYSML

```

package VehicleModel {

  part def Vehicle {
    attribute mass : ScalarValues::Real;
    part engine : Engine;
    part wheels : Wheel[4];
  }

  part def Engine {
    attribute power : ScalarValues::Real;
    part cylinders : Cylinder[1..12];
  }

  part def Cylinder;
  part def Wheel;

  part def Car :> Vehicle {
    attribute doors : ScalarValues::Integer;
  }

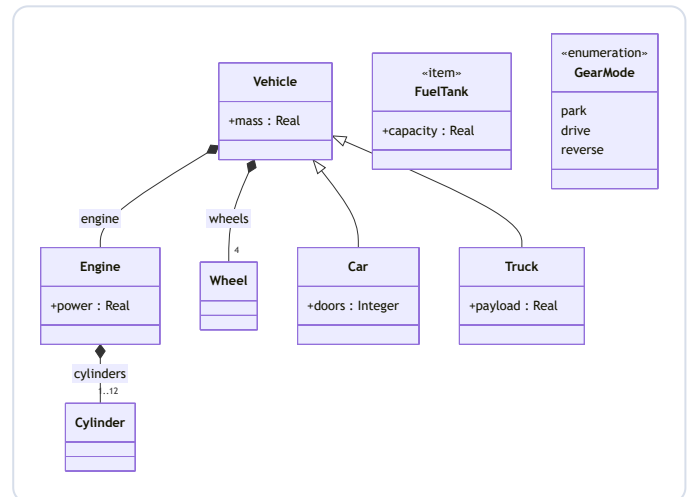
  part def Truck specializes Vehicle {
    attribute payload : ScalarValues::Real;
  }

  item def FuelTank {
    attribute capacity : ScalarValues::Real;
  }

  enum def GearMode {
    park;
    drive;
    reverse;
  }
}

```

CLASS DIAGRAM



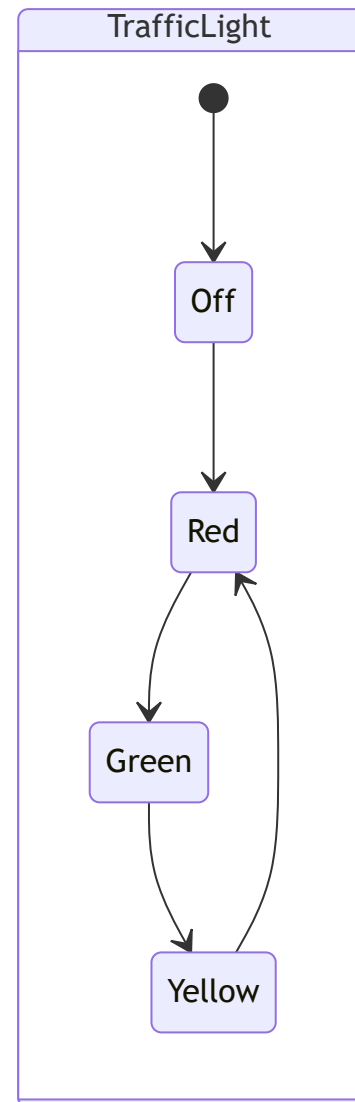
State — a traffic-light machine

A `state def` with its states and transitions. The `entry` pseudostate is rendered as the initial transition [*].

EXAMPLES/TRAFFIC_LIGHT.SYSML

```
package TrafficLightSM {  
    state def TrafficLight {  
        entry; then Off;  
  
        state Off;  
        state Red;  
        state Green;  
        state Yellow;  
  
        transition first Off then Red;  
        transition first Red then Green;  
        transition first Green then Yellow;  
        transition first Yellow then Red;  
    }  
}
```

STATE DIAGRAM



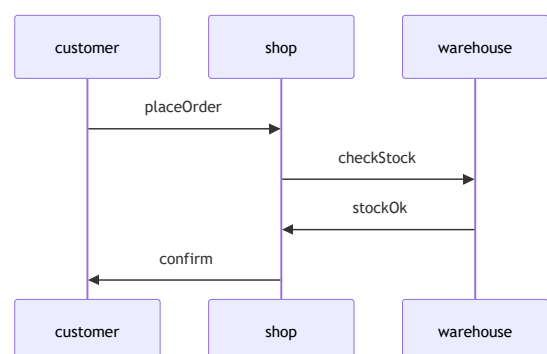
Sequence — message flow

Part usages become participants and directed `flows` become ordered messages between them.

EXAMPLES/ORDER_PROTOCOL.SYSML

```
package OrderProtocol {  
    part def Customer;  
    part def Shop;  
    part def Warehouse;  
  
    part customer : Customer;  
    part shop : Shop;  
    part warehouse : Warehouse;  
  
    flow placeOrder from customer to shop;  
    flow checkStock from shop to warehouse;  
    flow stockOk from warehouse to shop;  
    flow confirm from shop to customer;  
}
```

SEQUENCE DIAGRAM



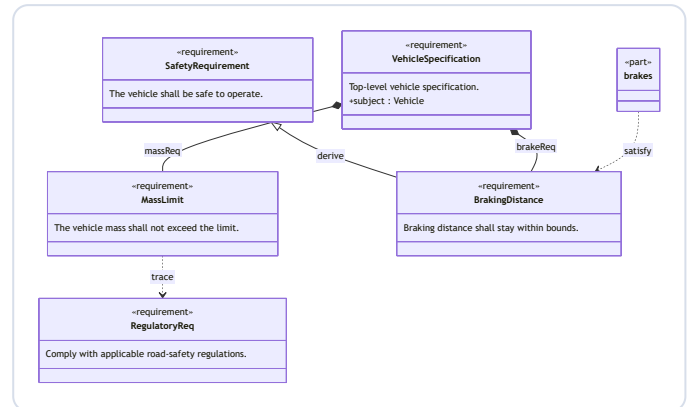
Requirement — derive, decompose & satisfy

Requirement definitions become «requirement» boxes. Specialization is derivation, nested requirement usages are decomposition, and a part that **satisfys** a requirement is linked with a satisfy dependency. The subject is shown inside the box.

EXAMPLES/VEHICLE_REQUIREMENTS.SYSML

```
package VehicleRequirements {  
  private import ScalarValues::*;  
  
  part def Vehicle;  
  part def Brakes;  
  
  requirement def RegulatoryReq {  
    doc /* Comply with applicable road-safety  
    regulations. */  
  }  
  
  requirement def SafetyRequirement {  
    doc /* The vehicle shall be safe to  
    operate. */  
  }  
  
  requirement def MassLimit {  
    doc /* The vehicle mass shall not exceed  
    the limit. */  
    attribute maxMass : Real;  
  }  
  
  requirement def BrakingDistance :>  
  SafetyRequirement {  
    doc /* Braking distance shall stay within  
    bounds. */  
  }  
  
  requirement def VehicleSpecification {  
    doc /* Top-level vehicle specification. */  
    subject vehicle : Vehicle;  
    requirement massReq : MassLimit;  
    requirement brakeReq : BrakingDistance;  
  }  
  
  // trace: the mass limit traces to a regulatory  
  requirement  
  dependency from MassLimit to RegulatoryReq;  
  
  part brakes : Brakes {  
    satisfy BrakingDistance;  
  }  
}
```

REQUIREMENT DIAGRAM



Class as BDD — function breakdown (FBS)

The class diagram is the SysML Block Definition Diagram. Each function is an **action def** (the «action» stereotype) and a function decomposed into sub-functions via typed **action** usages becomes composition — giving a top-down function breakdown tree.

EXAMPLES/BREAKDOWN/FUNCTION_BREAKDOWN.SYSML

```
// Function Breakdown Structure (FBS) - a
decomposition tree of functions.
//
// View as a Class diagram (right-click → Celeris
→ Show Class Diagram): the
// class diagram is the SysML Block Definition
Diagram (BDD). Each function is an
// 'action def'; a function is decomposed into sub-
functions via typed 'action'
// usages, which the BDD draws as composition
(whole-part). The result is a
// top-down breakdown tree of the mission's
functions.
```

```
package FunctionBreakdown {

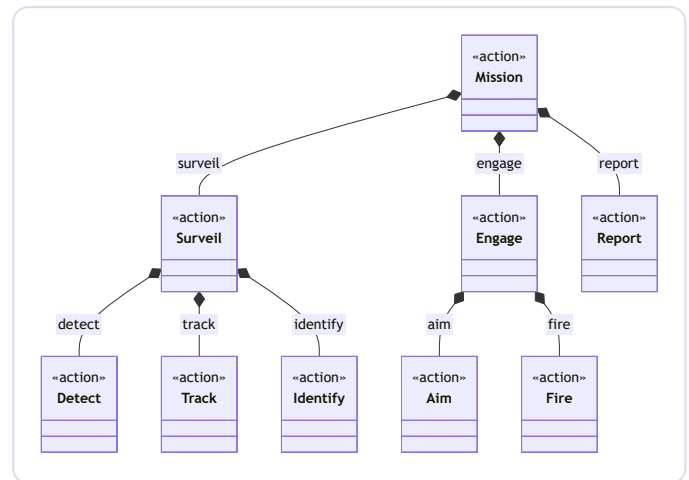
    action def Mission {
        action surveil : Surveil;
        action engage : Engage;
        action report : Report;
    }

    action def Surveil {
        action detect : Detect;
        action track : Track;
        action identify : Identify;
    }

    action def Engage {
        action aim : Aim;
        action fire : Fire;
    }

    // leaf functions (not decomposed further)
    action def Report;
    action def Detect;
    action def Track;
    action def Identify;
    action def Aim;
    action def Fire;
}
```

CLASS DIAGRAM



Class as BDD — component breakdown (PBS)

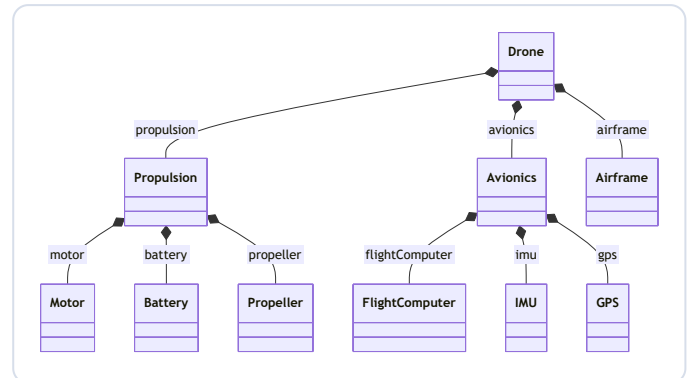
Same BDD, for structure: each component is a **part** def and typed **part** usages are composition, giving a top-down component breakdown tree.

EXAMPLES/BREAKDOWN/COMPONENT_BREAKDOWN.SYSML

```
// Component / Product Breakdown Structure (PBS) -  
a decomposition tree of components.  
//  
// View as a Class diagram (right-click → Celeris  
→ Show Class Diagram): the  
// class diagram is the SysML Block Definition  
Diagram (BDD). Each component is a  
// 'part def'; a component is decomposed into sub-  
components via typed 'part'  
// usages, which the BDD draws as composition  
(whole-part). The result is a  
// top-down breakdown tree of the drone's  
components.
```

```
package ComponentBreakdown {  
  
    part def Drone {  
        part airframe : Airframe;  
        part propulsion : Propulsion;  
        part avionics : Avionics;  
    }  
  
    part def Propulsion {  
        part motor : Motor;  
        part battery : Battery;  
        part propeller : Propeller;  
    }  
  
    part def Avionics {  
        part flightComputer : FlightComputer;  
        part imu : IMU;  
        part gps : GPS;  
    }  
  
    // leaf components (not decomposed further)  
    part def Airframe;  
    part def Motor;  
    part def Battery;  
    part def Propeller;  
    part def FlightComputer;  
    part def IMU;  
    part def GPS;  
}
```

CLASS DIAGRAM



Styling — Arcadia colour code

With the Arcadia style file selected, components are coloured by their Arcadia type, following the specialization hierarchy: operational entities orange, logical components blue, physical nodes yellow.

EXAMPLES/ARCADIA/DRONE_SYSTEM.SYSML

```
// Arcadia-styled example: a small drone
surveillance system.
// View as a Class diagram with celeris.styleFile
pointing at
// examples/arcadia/arcadia.style.json to get the
Arcadia colour code:
// LogicalComponent → blue, PhysicalNode →
yellow, OperationalEntity → orange,
// functions (actions) → green.
package DroneSystem {
  private import ArcadiaProfile::*;

  // --- Operational entities (orange) ---
  part def Operator :> OperationalEntity;
  part def AirspaceAuthority :>
OperationalEntity;

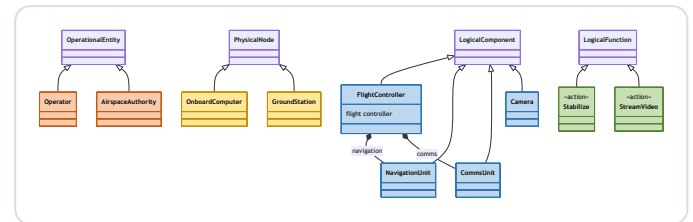
  // --- Logical components (blue) ---
  part def FlightController :> LogicalComponent {
    doc /* flight controller */
    part navigation : NavigationUnit;
    part comms : CommsUnit;
  }
  part def NavigationUnit :> LogicalComponent;
  part def CommsUnit :> LogicalComponent;
  part def Camera :> LogicalComponent;

  // --- Physical nodes (yellow) ---
  part def OnboardComputer :> PhysicalNode;
  part def GroundStation :> PhysicalNode;

  // --- Logical functions (green) ---
  action def Stabilize :> LogicalFunction;
  action def StreamVideo :> LogicalFunction;

  // --- System assembly ---
  part drone : FlightController {
    part eye : Camera;
  }
  part computer : OnboardComputer;
  part ground : GroundStation;
  part pilot : Operator;
}
```

CLASS DIAGRAM



Styling — NAF v4 by grid layer

With the NAF style file selected, elements are coloured by NAF grid layer: Concepts purple, Service blue, Logical green, Physical/Resource orange.

EXAMPLES/NAF/C2_SYSTEM.SYSML

```
// NAF v4 styled example: a small Command & Control
architecture spanning the
// NAF grid layers. View as a Class diagram with
the NAF style file selected
// (SysML Mermaid: Select Style File... →
examples/naf/naf.style.json):
// Capability → purple, Service → blue, Logical
→ green, Physical → orange.
package C2System {
    private import NafProfile::*;

    // --- Concepts layer: capabilities (purple) ---
    -
    part def SituationalAwareness :> Capability;
    part def CommandAndControl :> Capability;

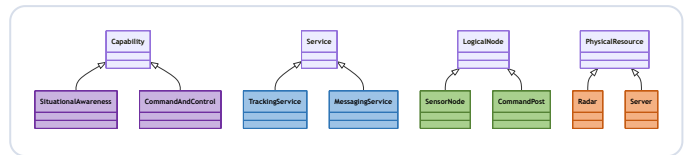
    // --- Service layer (blue) ---
    part def TrackingService :> Service;
    part def MessagingService :> Service;

    // --- Logical layer (green) ---
    part def SensorNode :> LogicalNode;
    part def CommandPost :> LogicalNode;

    // --- Physical / Resource layer (orange) ---
    part def Radar :> PhysicalResource;
    part def Server :> PhysicalResource;

    // --- Architecture assembly ---
    part c2 : CommandAndControl {
        part awareness : SituationalAwareness;
    }
    part tracking : TrackingService;
    part messaging : MessagingService;
    part post : CommandPost {
        part sensors : SensorNode[*];
    }
    part radar : Radar;
    part server : Server;
}
```

CLASS DIAGRAM



Interconnection (IBD) — structure

The SysML Internal Block Diagram. Each **part** is a block carrying its **ports**; a **connect** is a thin line and a typed **interface** a bold (purple) connector between ports.

EXAMPLES/INTERCONNECTION/AVIONICS_POWER.SYSML

```
// Interconnection (Internal Block Diagram) -
STRUCTURE.
//
// View as an Interconnection diagram (right-click
→ Celeris → Show
// Interconnection Diagram): each 'part' is a
block, its 'port's sit on the block,
// and 'connect' / 'interface' wire the ports
together. A typed 'interface' is
// drawn as a bold (purple) connector, a plain
'connect' as a thin line.
package AvionicsPower {

    port def PowerPort;
    port def DataPort;

    interface def PowerLink {
        end source : PowerPort;
        end sink : PowerPort;
    }

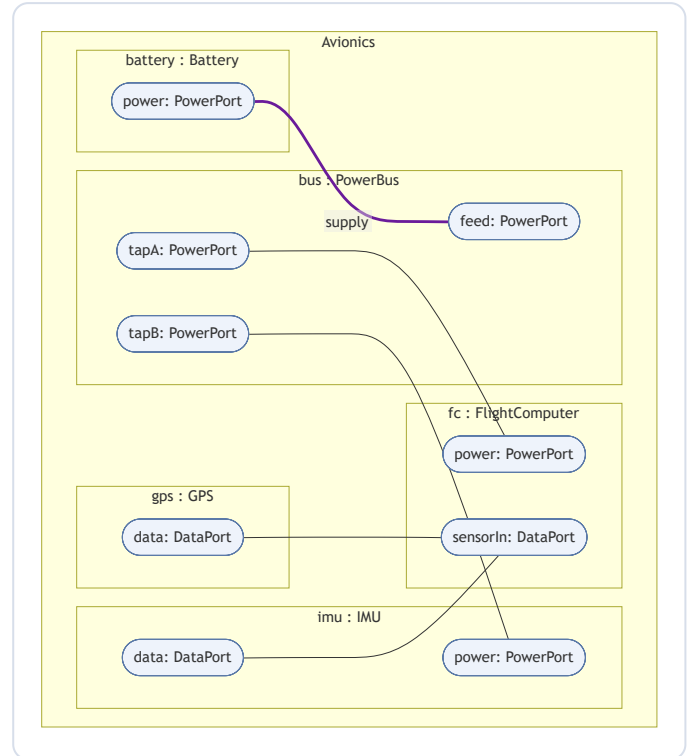
    part def Battery { port power : PowerPort; }
    part def PowerBus { port feed : PowerPort; port
tapA : PowerPort; port tapB : PowerPort; }
    part def FlightComputer { port power :
PowerPort; port sensorIn : DataPort; }
    part def IMU { port data : DataPort; port power
: PowerPort; }
    part def GPS { port data : DataPort; }

    part def Avionics {
        part battery : Battery;
        part bus : PowerBus;
        part fc : FlightComputer;
        part imu : IMU;
        part gps : GPS;

        // power distribution
        interface supply : PowerLink connect
battery.power to bus.feed;
        connect bus.tapA to fc.power;
        connect bus.tapB to imu.power;

        // data
        connect imu.data to fc.sensorIn;
        connect gps.data to fc.sensorIn;
    }
}
```

INTERCONNECTION DIAGRAM



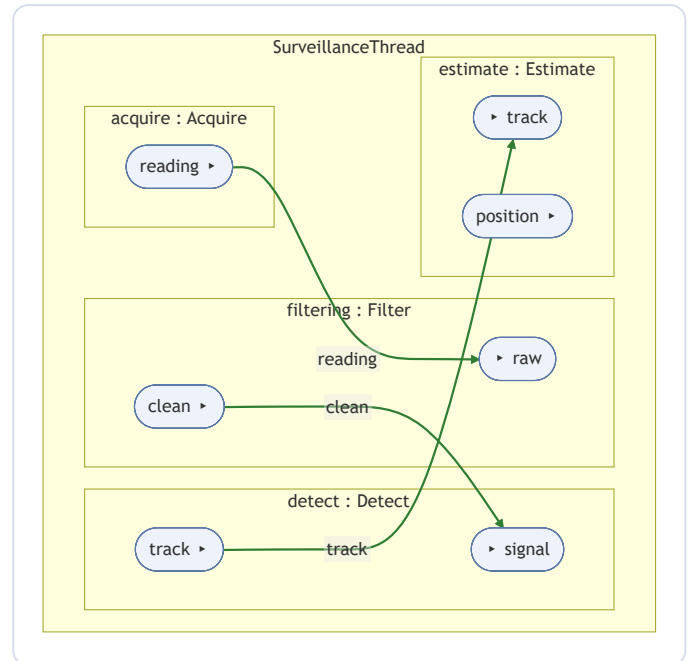
Interconnection (IBD) — data flow

Same view for behaviour: each **action** is a block carrying its **in/out ref** parameters (► marks the direction), and an **item flow** is a directed (green) connector labelled with the flowing item.

EXAMPLES/INTERCONNECTION/SIGNAL_CHAIN.SYSML

```
// Interconnection (Internal Block Diagram) -  
BEHAVIOUR / data flow.  
//  
// View as an Interconnection diagram (right-click  
→ Celeris → Show  
// Interconnection Diagram): each 'action' is a  
block, its 'in' / 'out' reference  
// parameters sit on the block (► marks the  
direction), and an item 'flow' between  
// an output and an input is drawn as a directed  
(green) connector labelled with  
// the item that flows.  
package SignalChain {  
  
    action def Acquire {  
        out ref reading;  
    }  
    action def Filter {  
        in ref raw;  
        out ref clean;  
    }  
    action def Detect {  
        in ref signal;  
        out ref track;  
    }  
    action def Estimate {  
        in ref track;  
        out ref position;  
    }  
  
    action def SurveillanceThread {  
        action acquire : Acquire;  
        action filtering : Filter;  
        action detect : Detect;  
        action estimate : Estimate;  
  
        flow from acquire.reading to filtering.raw;  
        flow from filtering.clean to detect.signal;  
        flow from detect.track to estimate.track;  
    }  
}
```

INTERCONNECTION DIAGRAM



Realization — traceability across engineering levels

Elements are placed in lanes by engineering level (set by specializing an Arcadia/NAF level type), most abstract on top; realization/traceability dependency links are drawn across the lanes (dashed, pointing to the element realized).

EXAMPLES/REALIZATION/CROSS_LEVEL.SYSML

```
// Realization view – traceability of how elements
// at one engineering level are
// realized by elements at another (Arcadia-style
// layers).
//
// View as a Realization diagram (right-click →
// Celeris → Show Realization
// Diagram): elements are placed in lanes by their
// level (an element sets its
// level by specializing a level marker type, e.g.
// `:> LogicalComponent`), and
// `dependency realize` / `dependency trace` links
// are drawn across the lanes.
package CrossLevelRealization {

    // --- engineering-level marker types (Arcadia
    // layers) ---
    part def OperationalEntity;
    action def OperationalActivity;
    action def SystemFunction;
    part def LogicalComponent;
    action def LogicalFunction;
    part def PhysicalNode;

    // --- Operational ---
    part def Operator :> OperationalEntity;
    action def ConductSurveillance :>
    OperationalActivity;

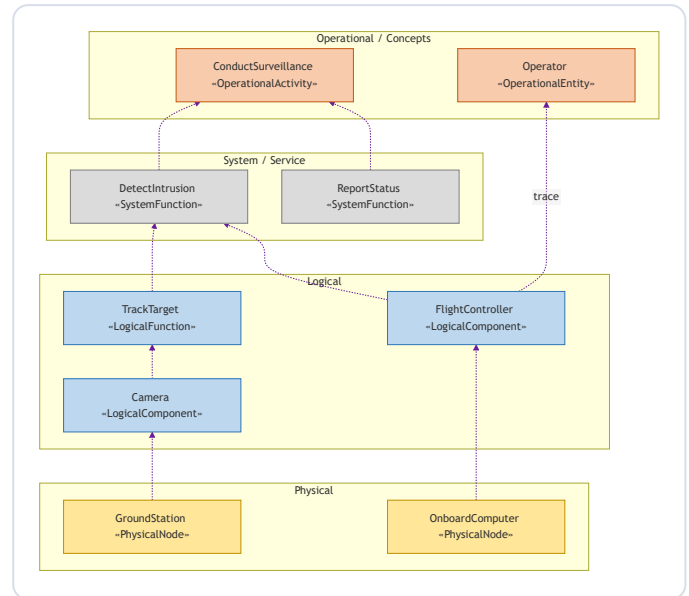
    // --- System ---
    action def DetectIntrusion :> SystemFunction;
    action def ReportStatus :> SystemFunction;

    // --- Logical ---
    part def FlightController :> LogicalComponent;
    part def Camera :> LogicalComponent;
    action def TrackTarget :> LogicalFunction;

    // --- Physical ---
    part def OnboardComputer :> PhysicalNode;
    part def GroundStation :> PhysicalNode;

    // --- Realization / traceability across levels
    // (lower realizes higher) ---
    // Plain `dependency` links are the realization
    // edges; a named dependency
    // (e.g. `trace`) is shown with its name as the
    // edge label.
    dependency from DetectIntrusion to
    ConductSurveillance; // System → Operational
    dependency from ReportStatus to
    ConductSurveillance; // System → Operational
    dependency from TrackTarget to DetectIntrusion;
    // Logical → System
    dependency from FlightController to
    DetectIntrusion; // Logical → System
    dependency from Camera to TrackTarget;
    // Logical → Logical
    dependency from OnboardComputer to
    FlightController; // Physical → Logical
    dependency from GroundStation to Camera;
    // Physical → Logical
    dependency trace from FlightController to
    Operator; // trace → Operational
}
```

REALIZATION DIAGRAM



Gantt — schedule, resources & critical path

An activity sequence (actions + successions) as a Gantt. **attribute duration** sets each bar's length, **attribute resource** its colour (one section per resource), and the critical path — the chain that fixes the total duration — is outlined in red.

EXAMPLES/GANTT/MISSION.SYSML

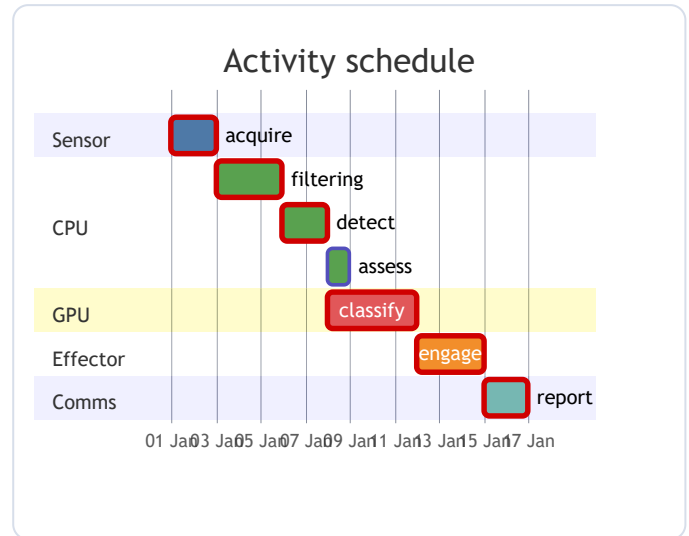
```
// Gantt (schedule) view. Right-click → Celeris →
// Show Gantt Diagram.
// Activities = actions; dependencies =
// successions. Each action carries planning
// attributes: 'duration' (days) sets the bar
// length, 'resource' sets its colour
// (one colour per resource, grouped into
// sections). The critical path - the chain
// that determines the total duration - is
// outlined in red.
package MissionSchedule {

    action def Mission {
        action acquire { attribute duration = 2;
        attribute resource = "Sensor"; }
        action filtering { attribute duration = 3;
        attribute resource = "CPU"; }
        action detect { attribute duration = 2;
        attribute resource = "CPU"; }
        action classify { attribute duration = 4;
        attribute resource = "GPU"; }
        action assess { attribute duration = 1;
        attribute resource = "CPU"; }
        action engage { attribute duration = 3;
        attribute resource = "Effector"; }
        action report { attribute duration = 2;
        attribute resource = "Comms"; }

        succession first acquire then filtering;
        succession first filtering then detect;
        succession first detect then classify;
        succession first detect then assess;
        succession first classify then engage;
        succession first assess then engage;
        succession first engage then report;

    }
}
```

GANTT DIAGRAM



Tabular view — requirements

The **Requirements** table preset: id, text, subject, derived-from and satisfied-by, one row per requirement. Right-click → Celeris → Show Table...; exportable to CSV.

EXAMPLES/VEHICLE_REQUIREMENTS.SYSML

```
package VehicleRequirements {
  private import ScalarValues::*;

  part def Vehicle;
  part def Brakes;

  requirement def RegulatoryReq {
    doc /* Comply with applicable road-safety regulations. */
  }

  requirement def SafetyRequirement {
    doc /* The vehicle shall be safe to operate. */
  }

  requirement def MassLimit {
    doc /* The vehicle mass shall not exceed the limit. */
    attribute maxMass : Real;
  }

  requirement def BrakingDistance :> SafetyRequirement {
    doc /* Braking distance shall stay within bounds. */
  }

  requirement def VehicleSpecification {
    doc /* Top-level vehicle specification. */
    subject vehicle : Vehicle;
    requirement massReq : MassLimit;
    requirement brakeReq : BrakingDistance;
  }

  // trace: the mass limit traces to a regulatory requirement
  dependency from MassLimit to RegulatoryReq;

  part brakes : Brakes {
    satisfy BrakingDistance;
  }
}
```

TABLE VIEW

Requirements (5 rows)

Id	Text	Subject	Derived from	Satisfied by
RegulatoryReq	Comply with applicable road-safety regulations.			
SafetyRequirement	The vehicle shall be safe to operate.			
MassLimit	The vehicle mass shall not exceed the limit.			
BrakingDistance	Braking distance shall stay within bounds.		SafetyRequirement	brakes
VehicleSpecification	Top-level vehicle specification.	Vehicle		

Tabular view — interfaces (ports / parts / flows)

The **Interfaces** table preset: each connect/interface with its source and target `part.port` and the item flows that traverse it. Built-in presets also cover requirements and any element type; custom tables via a `.table.json`.

EXAMPLES/TABLES/SYSTEM.SYSML

```
// Demo model for tabular views. Right-click → Celeris → Show Table...
// Presets: Requirements · Interfaces (ports / parts / flows) · Elements
// Custom tables: see celeris.table.json (set the 'celeris.tableConfig' setting to it).
package TabularDemo {

    // Engineering-level marker (used by the custom "Logical components" table).
    part def LogicalComponent;

    // — Requirements —
    requirement def SafetyReq { doc /* The system shall be safe to operate. */ }
    requirement def LatencyReq => SafetyReq {
        doc /* End-to-end latency shall stay under 100 ms. */
        subject sys : Controller;
    }
    requirement def AvailabilityReq { doc /* Availability shall be at least 99.9%. */ }

    // — Ports & interface —
    port def DataPort;
    interface def DataLink { end a : DataPort; end b : DataPort; }

    part def Sensor      :> LogicalComponent { port data : DataPort; }
    part def Controller  :> LogicalComponent { port sensorIn : DataPort; port cmdOut : DataPort; }
    part def Actuator    :> LogicalComponent { port cmd : DataPort; }

    // — Functions —
    action def Acquire { out ref reading; }
    action def Control { in ref reading; out ref command; }

    // — System: parts, interfaces and the flows that traverse them —
    part def System :> LogicalComponent {
        part sensor : Sensor;
        part controller : Controller;
        part actuator : Actuator;

        interface sensing : DataLink connect sensor.data to controller.sensorIn;
        interface actuation : DataLink connect controller.cmdOut to actuator.cmd;

        flow from sensor.data to controller.sensorIn;
        flow from controller.cmdOut to actuator.cmd;
    }
}
```

TABLE VIEW

Interfaces (2 rows)

Interface	Kind	Source (part.port)	Target (part.port)	Item flows
sensing	interface	sensor.data	controller.sensorIn	data
actuation	interface	controller.cmdOut	actuator.cmd	cmdOut